Projeto IoT — Crachá Inteligente de Ponto (UNIFESP)

Oct 27, 2026 · @Pedro Henrique Sampaio Coelho

Visão geral

O funcionário bate o ponto de dois jeitos com o mesmo crachá (uma placa Heltec LoRa 32): **apertando um botão**, o crachá envia o ponto por LoRa até um gateway, mesmo a centenas de metros; e, **ao chegar perto do computador**, o PC detecta o crachá por Bluetooth (BLE), bate o ponto automático e desbloqueia a tela. Os dois caminhos chegam ao Node-RED via MQTT, são gravados no PostgreSQL e aparecem num painel de gerência.

Revisão de 29/10 (após conversa com o professor): o crachá passou a ser a própria Heltec com botão, LED, buzzer e relógio DS1302; o nó de mesa com ESP32 + RFM95W saiu do projeto; a segunda Heltec é o gateway, conectado à internet pelo hotspot do celular ou pelo roteador da faculdade.

Item obrigatório / pontuação	Onde aparece no projeto
Levantamento de requisitos	Seção Requisitos deste documento
Git	Repositório único com firmware do crachá e do gateway, agente do PC, fluxos do Node-RED e PCI
Validação de circuito (1 pt)	Protoboard: botão → LoRa → gateway → MQTT, com LED, buzzer e relógio funcionando
Nuvem com MQTT + Node-RED (2 pts)	Mosquitto + Node-RED + painel
Comunicação LoRa	Crachá ↔ gateway, ponto-a-ponto em 915 MHz
PCI + documentação (3 pts)	Placa-base onde a Heltec encaixa, com botão, LED, buzzer e relógio (mesmo desenho para crachá e gateway)
Apresentação com evidências	Vídeos, prints do painel, display OLED mostrando os eventos, fotos da placa
Extras (+3)	Banco PostgreSQL, OTA via Wi-Fi, caixas impressas em 3D

Prazo: de 27/10 a 04/12 são cerca de 5 semanas e meia. Cada termo técnico usado aqui está explicado na seção Conceitos básicos logo abaixo, e cada decisão vem com o motivo ("Por quê"), para que todos consigam defender o projeto na arguição.

Conceitos básicos

Em palavras simples: o crachá "grita" seu nome por rádio o tempo todo; o aparelho na mesa "ouve" esse grito e, pelo volume, sabe se a pessoa está perto ou longe; quando muda, ele avisa o computador e manda um recado por rádio de longo alcance até uma central, que entrega o recado à internet, onde um programa anota tudo e monta o painel.

Termo	O que é	Onde aparece no projeto
IoT (Internet das Coisas)	Objetos físicos com sensores e rádio que trocam dados entre si e com a internet, sem alguém digitando	O sistema inteiro
ESP32	Microcontrolador (um "mini computador" num chip) com Wi-Fi e Bluetooth embutidos, programável pela Arduino IDE	Heltec do crachá e do gateway
Firmware	O programa gravado dentro do microcontrolador; roda sozinho assim que a placa liga	Um firmware para cada tipo de placa
BLE (Bluetooth Low Energy)	Versão do Bluetooth feita para gastar pouca bateria. Um dispositivo pode só "anunciar" (advertising) pacotes curtos sem precisar parear	O crachá anuncia; o computador escuta (scan)
RSSI	"Received Signal Strength Indicator": a força do sinal recebido, em dBm. É negativo: −50 dBm é forte (perto), −90 dBm é fraco (longe)	Para estimar se o crachá está perto do PC
Histerese	Usar dois limites diferentes para ligar e desligar, para o sistema não ficar "piscando" quando o valor oscila perto de um limite só (como o termostato da geladeira)	Bloqueio em −75 dBm, desbloqueio em −65 dBm

Termo	O que é	Onde aparece no projeto
LoRa	"Long Range": modulação de rádio que alcança centenas de metros a quilômetros com pouca energia, em troca de transmitir poucos dados por vez	Crachá ↔ gateway
LoRaWAN	Um protocolo de rede completo por cima do LoRa (com servidores de rede, chaves etc.). Não vamos usar: faremos LoRa ponto-a-ponto, mais simples de entender e depurar	—
915 MHz	Frequência de rádio do LoRa. No Brasil a Anatel permite o uso livre na faixa de 915 a 928 MHz; módulos de 433 MHz ou 868 MHz são de outras regiões	Heltec LoRa 32 V3
SF (Spreading Factor)	Parâmetro do LoRa entre 7 e 12: quanto maior, mais alcance, mas mais lento	Começamos em SF7
Gateway (concentrador)	Aparelho que fica no meio de duas redes e traduz uma para a outra; aqui recebe LoRa e reenvia por Wi-Fi	Heltec LoRa 32
ACK	"Acknowledgement": a confirmação "recebi". Se quem enviou não recebe o ACK, manda de novo	Gateway → nó a cada evento
Topologia em estrela	Formato de rede em que vários dispositivos falam com um centro, e não entre si	Crachás → 1 gateway
MQTT	Protocolo leve de mensagens da IoT no modelo publicar/assinar: quem envia "publica" num tópico (um endereço em texto) e quem tem interesse "assina" aquele tópico	Gateway ↔ servidor
Broker	O "correio" do MQTT: recebe as mensagens publicadas e entrega para quem assinou o tópico	Mosquitto
JSON	Formato de texto para dados estruturados, fácil de ler por pessoas e programas: `{"badge":101,"tipo":"CHEGADA"}`	Mensagens MQTT

Termo	O que é	Onde aparece no projeto
Node-RED	Ferramenta de programação visual: você liga "caixinhas" (nós) com fios para montar o fluxo dos dados, sem escrever um servidor do zero. Também gera painéis (dashboard)	Regras, cálculo de jornada e painel
Docker	Ferramenta que roda programas dentro de "containers", pacotes isolados que já vêm configurados. Com um arquivo `docker-compose.yml` sobe-se tudo com um comando	Mosquitto, Node-RED e PostgreSQL
PostgreSQL	Banco de dados relacional: guarda os dados em tabelas ligadas entre si e responde consultas em SQL	Histórico de eventos e jornadas
SQL	Linguagem para criar tabelas e consultar dados: `SELECT * FROM evento WHERE badge_id = 101;`	Consultas do Node-RED ao banco
NTP	Protocolo que acerta o relógio de um dispositivo pela internet	Gateway carimba a hora dos eventos
OTA (Over-The-Air)	Atualizar o firmware sem cabo, pela rede	Desafio extra
UART / serial USB	Comunicação simples por texto entre a placa e o PC pelo cabo USB (a mesma do Monitor Serial da Arduino IDE)	Gravação e Monitor Serial das Heltec
SPI	Barramento de 4 fios que o microcontrolador usa para conversar com outros chips rapidamente	ESP32 ↔ módulo LoRa
PCI (PCB)	Placa de Circuito Impresso: as ligações da protoboard viram trilhas de cobre numa placa definitiva	Placa-base da Heltec
KiCad	Programa gratuito para desenhar o esquemático e o layout da PCI e gerar os arquivos de fabricação (Gerber)	Projeto da PCI
Git / GitHub	Git guarda o histórico de cada alteração do código (commits); o GitHub hospeda o repositório para o grupo trabalhar junto	Item obrigatório

Levantamento de requisitos

O escopo de demonstração é 1 crachá, 1 gateway, 1 computador de funcionário e 1 servidor, usando as 2 placas Heltec que o grupo já tem. Para ter 2 crachás seria preciso uma terceira placa LoRa. O computador do funcionário e o servidor podem ser o mesmo notebook.

Requisitos funcionais

ID	Requisito	Prioridade
RF01	Ao apertar o botão do crachá, ele envia por LoRa um evento de ponto com ID do funcionário, data e hora	Obrigatório
RF02	O gateway confirma (ACK) cada evento; sem ACK o crachá retransmite	Obrigatório
RF03	O crachá dá retorno ao funcionário: bipe + LED quando o ponto foi confirmado, sinal diferente em caso de erro	Obrigatório
RF04	O crachá guarda data e hora num relógio (DS1302) para carimbar o evento mesmo se estiver longe do gateway	Obrigatório
RF05	O crachá anuncia seu ID por BLE continuamente	Obrigatório
RF06	O computador detecta o crachá por BLE e, quando ele se aproxima, bate o ponto automático e desbloqueia a tela	Obrigatório
RF07	Quando o crachá se afasta do computador, a tela é bloqueada	Obrigatório
RF08	O gateway e o computador publicam os eventos no broker MQTT em JSON	Obrigatório
RF09	O Node-RED identifica a origem do ponto (botão/LoRa ou automático/BLE), calcula as horas do dia e compara com a carga horária	Obrigatório
RF10	Painel de gerência: quem está presente, horas do dia/semana, saldo, histórico	Obrigatório
RF11	Histórico persistido em banco de dados	Extra (desafio)

ID	Requisito	Prioridade
RF12	Atualização de firmware remota (OTA) do crachá e do gateway	Extra (desafio)
RF13	Ligar o computador à distância (Wake-on-LAN) quando o funcionário bate o ponto	Desejável, depende da rede do laboratório
RF14	Display OLED do crachá mostra hora e status do último ponto; o do gateway mostra o último evento recebido	Desejável

Requisitos não funcionais

ID	Requisito	Meta
RNF01	Tempo entre apertar o botão e o bipe de confirmação	⩽ 2 s
RNF02	Tempo entre o crachá se aproximar e o desbloqueio	⩽ 5 s
RNF03	Tempo entre o crachá se afastar e o bloqueio	⩽ 10 s
RNF04	Evento chegar ao painel	⩽ 5 s
RNF05	Alcance LoRa crachá ↔ gateway	todo o prédio do laboratório (testar e registrar a distância)
RNF06	Nenhum ponto perdido se o gateway estiver fora do alcance (fila no crachá)	0 perdas
RNF07	Ponto contado uma vez só, mesmo com retransmissão	0 duplicatas
RNF08	Erro do relógio do crachá	< 2 s (acertado pelo gateway a cada ACK)
RNF09	Frequência LoRa permitida pela Anatel	915–928 MHz
RNF10	Código versionado em Git com commits de cada membro	Obrigatório

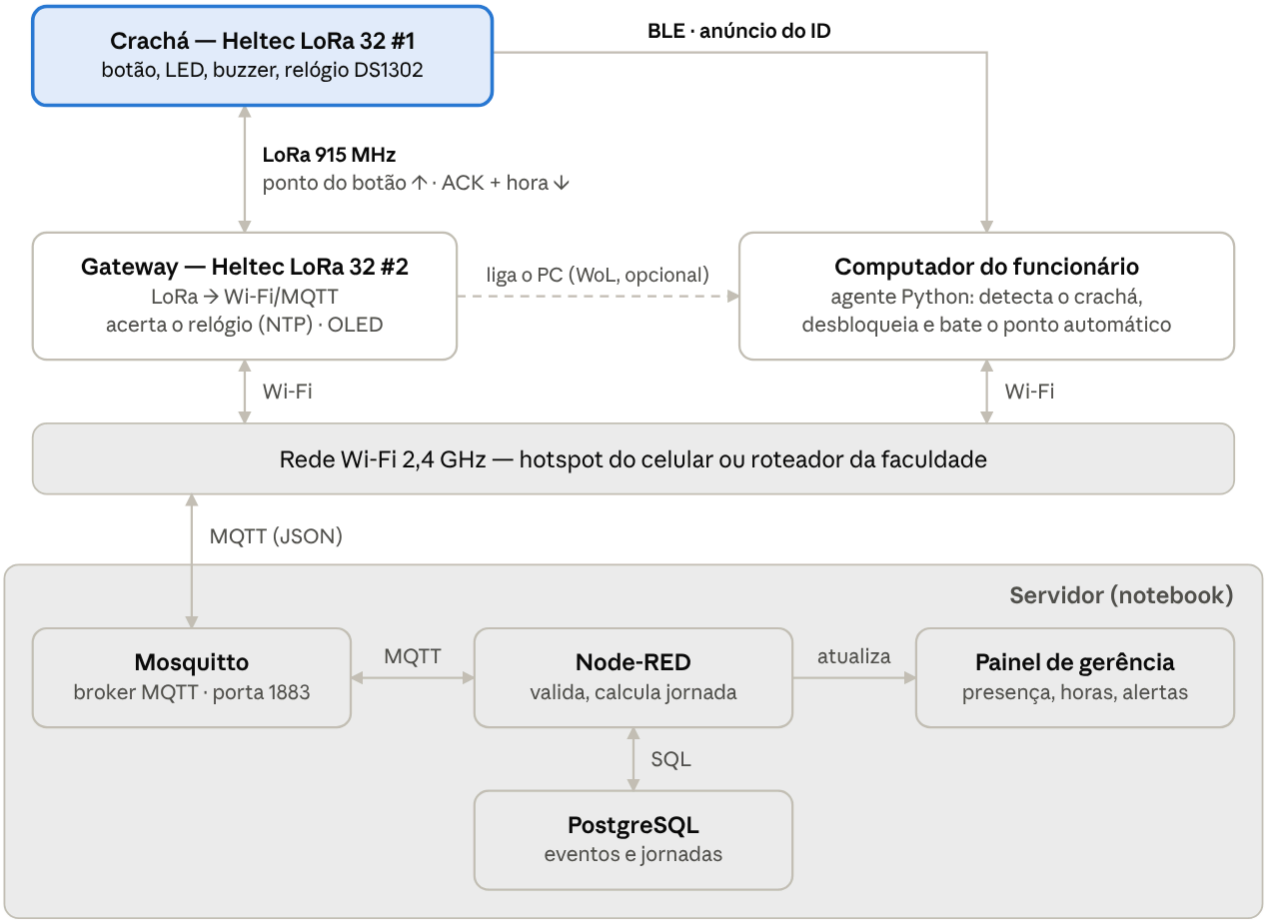
Restrições

- Hardware: 2 Heltec WiFi LoRa 32 V3 (já temos), módulo de relógio DS1302, botões, LEDs, buzzers.
- O crachá só tem BLE (Bluetooth Low Energy), sem Bluetooth Clássico: o computador precisa ter Bluetooth 4.0 ou mais novo (ver teste de compatibilidade em "Como cada bloco funciona").
- Placas alimentadas por USB durante os testes.
- Internet do gateway: hotspot do celular ou roteador da faculdade, sempre em 2,4 GHz (o ESP32 não enxerga redes de 5 GHz).
- Projeto acadêmico: a segurança do crachá é simplificada (lista de IDs autorizados).

Arquitetura e topologia da rede

O sistema tem dois caminhos de rádio saindo do mesmo crachá e uma rede Wi-Fi que leva tudo ao servidor: LoRa (longo alcance) do crachá até o gateway, BLE (curto alcance) do crachá até o computador, e Wi-Fi/MQTT do gateway e do computador até o servidor.

O mesmo crachá bate o ponto por LoRa (botão) e por BLE (perto do PC)



arquitetura e topologia · 3 camadas, 4 enlaces

Atenção à diferença entre gateway e roteador. O *gateway* é a segunda Heltec: é ela que recebe o LoRa, porque um roteador Wi-Fi ou um celular não têm rádio LoRa. O hotspot do celular ou o roteador da faculdade são o *ponto de acesso Wi-Fi*: dão acesso à rede para o gateway, o computador e o servidor se encontrarem. O projeto precisa dos dois.

Recomendação: hotspot do celular. Redes de universidade costumam usar login com usuário e senha (WPA2-Enterprise/eduroam), página de login no navegador ou isolamento entre aparelhos, e qualquer um deles impede que o ESP32 converse com o notebook. No hotspot vocês controlam tudo. No iPhone, ativem "Maximizar compatibilidade" para forçar 2,4 GHz; no Android, escolham a banda de 2,4 GHz. Todos os aparelhos (gateway, computador e servidor) devem estar na mesma rede.

Diagrama das comunicações

O diagrama de arquitetura mostra *quem fala com quem*; este mostra *em que ordem* as mensagens acontecem. É um **diagrama de sequência**: cada coluna é um participante, a linha tracejada vertical abaixo dele é a sua "linha do tempo", e cada seta horizontal é uma mensagem. Lendo de cima para baixo, você segue uma batida de ponto do crachá até o banco de dados e, depois, um comando fazendo o caminho de volta.

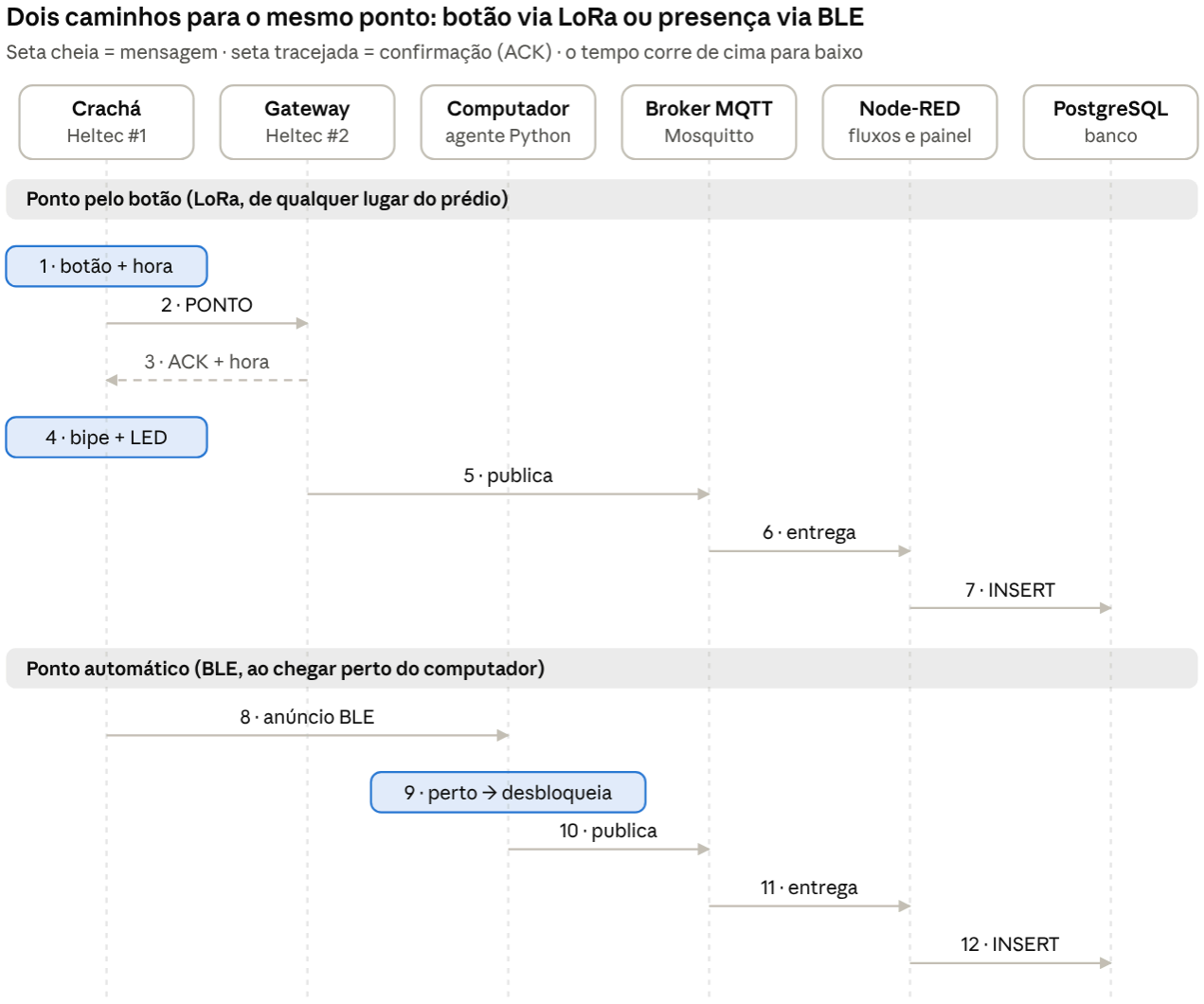


diagrama de sequência · chegada e bloqueio remoto, 14 passos

O que acontece em cada passo

Passo	De → Para	Meio	O que acontece
1	Crachá (interno)	—	O funcionário aperta o botão; o crachá lê a data e a hora no relógio DS1302 e monta o pacote

Passo	De → Para	Meio	O que acontece
2	Crachá → Gateway	LoRa 915 MHz	Pacote binário pequeno: tipo PONTO, crachá 101, seq 42, hora do evento
3	Gateway → Crachá	LoRa	ACK com o mesmo seq 42 e a hora certa (que o gateway pega da internet por NTP). O crachá usa essa hora para acertar o relógio. Sem ACK em 2 s, o crachá volta ao passo 2 (até 3 vezes); depois guarda o ponto na fila e tenta de novo a cada 30 s
4	Crachá (interno)	—	Bipe curto + LED aceso = ponto confirmado. Três bipes = falhou, está na fila
5	Gateway → Broker	Wi-Fi + MQTT	O gateway converte para JSON e publica em `unifesp/ponto/evento` com `"origem":"botao"`
6	Broker → Node-RED	MQTT	O Mosquitto entrega a mensagem a quem assinou o tópico: o Node-RED
7	Node-RED → PostgreSQL	SQL	`INSERT` na tabela `evento` e recálculo da jornada do dia; o painel é atualizado
8	Crachá → Computador	BLE	O crachá anuncia seu ID a cada 500 ms, o tempo todo; o computador mede a força do sinal (RSSI)
9	Computador (interno)	—	Sinal forte por 3 s = funcionário chegou: o agente desbloqueia a tela. Sinal fraco ou sumido por 8 s = saiu: bloqueia
10	Computador → Broker	Wi-Fi + MQTT	O agente publica o mesmo tópico com `"origem":"ble"` (ponto automático)
11	Broker → Node-RED	MQTT	Mesma entrega do passo 6
12	Node-RED → PostgreSQL	SQL	Mesmo registro do passo 7, com a origem diferente

Por que dois caminhos? O botão com LoRa permite bater o ponto longe do computador (na entrada do prédio, no laboratório vizinho), e é onde o LoRa mostra sua vantagem de alcance. O BLE resolve o caso de quem trabalha no computador: o ponto e o desbloqueio acontecem sozinhos. O Node-RED recebe os dois pelo mesmo tópico e só olha o campo `origem` para saber de onde veio.

E se o funcionário usar os dois? Apertar o botão na entrada e depois chegar ao computador geraria duas CHEGADAS. A regra no Node-RED: um segundo ponto do mesmo tipo em menos de 30 minutos é registrado no histórico, mas não conta para a jornada.

Ferramentas: Arduino IDE + ESP32

Sim, dá para fazer o projeto inteiro no Arduino: vamos programar as placas ESP32 pela Arduino IDE, com a mesma linguagem (`setup()` e `loop()`) e o mesmo Monitor Serial que vocês já usam em aula. O ESP32 é compatível com o "mundo Arduino"; só é preciso instalar o pacote dele na IDE.

Por que ESP32 programado no Arduino, e não uma placa Arduino Uno? O Uno não tem Bluetooth nem Wi-Fi, tem apenas 2 KB de memória RAM e trabalha em 5 V, enquanto o módulo LoRa trabalha em 3,3 V. Para fazer o mesmo com Uno seriam necessários módulos extras (Bluetooth HM-10, Wi-Fi ESP-01, conversor de nível), mais fios e mais pontos de falha. O ESP32 já traz tudo isso num chip e custa menos.

Como configurar (uma vez em cada computador)

1. Instalar a Arduino IDE 2.x.
2. Em Arquivo → Preferências → "URLs adicionais do Gerenciador de Placas", colar o endereço do pacote da Espressif: `https://espressif.github.io/arduino-esp32/package_esp32_index.json` .
3. Em Ferramentas → Placa → Gerenciador de Placas, instalar "esp32 by Espressif Systems".
4. Instalar as bibliotecas pelo Gerenciador de Bibliotecas (tabela abaixo).
5. Para cada placa, escolher em Ferramentas a placa certa e o esquema de partição "Minimal SPIFFS (1.9MB APP with OTA)" — explicado no desafio de OTA.

Placa	Opção em Ferramentas → Placa	Observação
Crachá e gateway (Heltec WiFi LoRa 32 V3)	"Heltec WiFi LoRa 32(V3)", que vem no pacote esp32 da Espressif	Se a gravação não começar, segure o botão PRG, aperte e solte RST, solte PRG. A caixa diz 863~928 MHz porque o rádio cobre várias regiões; a frequência de 915 MHz é escolhida no código

As duas placas são iguais; o que muda é o firmware gravado em cada uma (`firmware/cracha` ou `firmware/gateway`).

Bibliotecas

Biblioteca	Para que serve	Usada em
RadioLib	Controla o rádio LoRa da Heltec V3 (chip SX1262)	Crachá e gateway
NimBLE-Arduino	Bluetooth Low Energy; ocupa bem menos memória que a biblioteca BLE padrão	Crachá
Rtc by Makuna	Lê e acerta o relógio DS1302 (classe `RtcDS1302`)	Crachá
ESP8266 and ESP32 OLED driver for SSD1306 (ThingPulse)	Escreve no display OLED da Heltec	Crachá e gateway
PubSubClient	Cliente MQTT: conecta no broker, publica e assina tópicos	Gateway
ArduinoJson	Monta e lê mensagens JSON	Gateway
WiFi, HTTPUpdate, ArduinoOTA	Já vêm com o pacote do ESP32: Wi-Fi, hora por NTP e atualização remota	Crachá e gateway
bleak e paho-mqtt (Python, via `pip install`)	BLE e MQTT no computador	Agente do PC

Atenção com o display: na Heltec V3 o OLED só liga depois que o pino Vext (GPIO 36) é colocado em nível baixo no `setup()`; sem isso a tela fica preta mesmo com o código certo.

No Git, cada firmware é uma pasta de sketch (a pasta precisa ter o mesmo nome do arquivo `.ino`), por exemplo `firmware/cracha/cracha.ino`. Valores que mudam entre placas (ID do crachá, senha do Wi-Fi) ficam num arquivo `config.h` separado; a senha real não vai para o GitHub (usar um `config.example.h`).

Como cada bloco funciona

São 5 blocos: crachá, gateway, computador do funcionário, rede Wi-Fi e servidor. As duas Heltec são placas idênticas; o papel de cada uma é definido só pelo firmware gravado nela.

1. Crachá (Heltec LoRa 32 #1)

O que faz: bate o ponto pelo botão (via LoRa) e fica anunciando "eu sou o funcionário 101" por BLE para o computador.

- **Hardware:** Heltec V3 + botão + LED + buzzer + módulo de relógio DS1302 com bateria CR2032, alimentados pelo cabo USB-C.
- **Ao apertar o botão:**
  1. O firmware ignora repiques do botão por 50 ms (*debounce*: o contato metálico "quica" e geraria vários cliques).
  2. Lê data e hora no DS1302 e monta o pacote PONTO.
  3. Envia por LoRa e espera o ACK por 2 s.
  4. Com ACK: 1 bipe + LED aceso por 1 s, e acerta o relógio com a hora que veio no ACK. Sem ACK: tenta 3 vezes; se continuar sem resposta, guarda na fila, dá 3 bipes e reenvia a cada 30 s.
- **O crachá não decide se é chegada ou saída:** ele só diz "bati o ponto". O Node-RED alterna: se o último registro do dia foi CHEGADA, o novo é SAIDA, e vice-versa. Assim o crachá precisa de um botão só.
- **Ao mesmo tempo, o BLE:** a cada 500 ms o crachá anuncia o nome `UNIFESP-BADGE` com o ID do funcionário, sem precisar parear com ninguém.
- **Display OLED:** mostra a hora e o resultado do último ponto ("Ponto OK 08:02" ou "Na fila: 1").

Por que um relógio DS1302 se o ESP32 tem relógio interno? O relógio interno do ESP32 zera quando a placa é desligada e só é acertado com internet, que o crachá não tem. O DS1302 tem a própria bateria e continua contando o tempo mesmo com a placa desligada. Assim, um ponto batido fora do alcance do gateway fica com a hora certa enquanto espera na fila.

Como o DS1302 se liga: ele usa 3 fios de dados próprios (CLK, DAT e RST), além de VCC (3,3 V) e GND. Não é I²C, então pode ir em quaisquer 3 pinos livres. Ele funciona de 2 a 5,5 V, então liga direto no 3,3 V da Heltec.

2. Gateway (Heltec LoRa 32 #2)

O que faz: recebe o LoRa do crachá e entrega no servidor pela rede Wi-Fi.

- Conecta no Wi-Fi (hotspot) e acerta o próprio relógio pela internet (NTP).
- Ao receber um pacote: confere se está íntegro e se o crachá é conhecido, responde ACK com a hora certa, converte para JSON e publica no MQTT.
- Mostra no OLED o último ponto recebido e a qualidade do sinal (RSSI/SNR): ótima evidência ao vivo na apresentação.

- O segundo conjunto de LED + buzzer + botão vai no gateway: LED pisca e buzzer apita a cada ponto recebido; o botão serve para testes.
- **Opcional (ligar o PC):** ao receber a primeira CHEGADA do dia, o gateway pode enviar um *Wake-on-LAN*, um pacote especial na rede que liga o computador. Funciona só se o PC estiver no cabo de rede, na mesma rede do gateway e com a opção ativada na BIOS; notebooks no Wi-Fi geralmente não aceitam. Testar no laboratório antes de prometer.

3. Computador do funcionário (agente Python)

O que faz: um programa no computador detecta o crachá pelo Bluetooth do próprio PC, bloqueia/desbloqueia a tela e envia o ponto automático.

- Python com `bleak` (BLE) e `paho-mqtt` (MQTT). O agente procura anúncios com o nome `UNIFESP-BADGE` e mede o RSSI.
- Mesma lógica de presença de antes: média das 5 últimas leituras; PRESENTE se acima de −65 dBm por 3 s; AUSENTE se abaixo de −75 dBm ou sumido por 8 s (valores a calibrar).
- Na mudança de estado: desbloqueia ou bloqueia a tela e publica o ponto com `"origem":"ble"`.
- Linux: `loginctl lock-session` / `unlock-session` fazem bloqueio e desbloqueio reais. Windows: só o bloqueio é permitido (`LockWorkStation`); para "desbloquear", o agente usa uma tela de bloqueio própria em tela cheia.

Teste de compatibilidade do Bluetooth (fazer nesta semana): o crachá só fala BLE, então o computador precisa de Bluetooth 4.0 ou mais novo.

1. **Windows:** Gerenciador de Dispositivos → Bluetooth → botão direito no adaptador → Propriedades → aba Avançado → linha "Firmware" ou "LMP". LMP 6 = Bluetooth 4.0, 7 = 4.1, 8 = 4.2, 9 = 5.0, 10 ou mais = 5.1+. Precisa ser 6 ou mais. Se não existir a categoria Bluetooth, o PC não tem Bluetooth.
2. **Linux:** no terminal, `hciconfig -a` mostra "HCI Version" (precisa ser 4.0 ou mais); `bluetoothctl show` confirma se o adaptador existe.
3. **Teste definitivo:** com o crachá ligado anunciando BLE, rodar o script abaixo. Se o crachá aparecer na lista, o computador é compatível.

```
# pip install bleak
import asyncio
from bleak import BleakScanner

async def main():
    achados = await BleakScanner.discover(timeout=10, return_adv=True)
    for dispositivo, anuncio in achados.values():
        print(dispositivo.address, dispositivo.name, anuncio.rssi, "dBm")

asyncio.run(main())
```

Se o PC não tiver Bluetooth ou for anterior ao 4.0, a solução é um **adaptador USB Bluetooth 5.0** (pen-drive pequeno, cerca de R$ 30–50), que funciona no Windows e no Linux.

4. Rede Wi-Fi (hotspot do celular ou roteador da faculdade)

O que faz: é a "estrada" por onde gateway, computador e servidor se encontram. Não processa nada do projeto. Precisa ser 2,4 GHz, e todos os aparelhos precisam estar conectados à mesma rede (ver a recomendação na seção de arquitetura).

5. Servidor: onde entra o Node-RED

O que faz: o servidor é um notebook que roda três programas: Mosquitto (o correio MQTT), **Node-RED** (o "cérebro" que processa os pontos) e PostgreSQL (o arquivo permanente).

O que é o Node-RED: uma ferramenta em que vocês montam o programa do servidor ligando "caixinhas" (chamadas nós) com fios, no navegador, em vez de escrever um servidor do zero. Cada caixinha faz uma tarefa, e a mensagem passa de uma para a outra seguindo os fios. Esse desenho é chamado de **fluxo**.

Onde ele entra: em tudo o que acontece depois que a mensagem chega ao broker MQTT. O gateway e o computador só *avisam* que houve um ponto; quem decide o que fazer com o aviso é o Node-RED. Ele é a "etapa de nuvem" exigida pelo professor (item obrigatório 3) e vale os 2 pontos de validação de nuvem com MQTT.

No nosso projeto o Node-RED tem 5 tarefas:

1. **Receber:** um nó `mqtt in` assina `unifesp/ponto/evento` e recebe os pontos do botão e do BLE.
2. **Interpretar:** um nó `json` transforma o texto em dados, e um nó `function` (pequeno trecho de JavaScript) decide se é CHEGADA ou SAIDA, descarta duplicatas e calcula as horas do dia.
3. **Guardar:** um nó `postgresql` executa o `INSERT` no banco.

4. **Mostrar:** nós de dashboard (tabela, medidor, gráfico) montam o painel de gerência, aberto em `http://<ip-do-notebook>:1880/dashboard`.
5. **Comandar:** um botão do painel liga em um nó `mqtt out`, que publica comandos de volta (por exemplo, iniciar o OTA do crachá).

O fluxo principal fica assim:

```
[mqtt in: unifesp/ponto/evento] → [json] → [function: chegada ou saída?] →
[postgresql: INSERT] → [function: calcula horas] → [dashboard: tabela +
gráfico]
```

Como instalar: pelo Docker (junto com Mosquitto e PostgreSQL no mesmo `docker-compose.yml`) ou direto com Node.js (`npm install -g node-red` e depois `node-red`). O editor abre em `http://localhost:1880`. Os fluxos são exportados como `flows.json` e versionados no Git.

Por que o Node-RED e não um programa escrito do zero? Ele já tem prontos os nós de MQTT, banco de dados e painel; vocês só escrevem a regra de negócio (as funções). O fluxo desenhado também serve de diagrama na apresentação: o professor vê o caminho da mensagem.

Protocolos e dados

Um protocolo é o "combinado" de como as mensagens são escritas, para que quem envia e quem recebe se entendam. Definir estes três contratos (pacote LoRa, tópicos MQTT, tabelas do banco) na semana 1 permite que as 4 pessoas trabalhem em paralelo: cada uma programa seu bloco seguindo o combinado, sem esperar a outra terminar.

Enlace LoRa (ponto-a-ponto, sem LoRaWAN)

Configuração de rádio, igual em todas as placas (se um parâmetro for diferente, as placas simplesmente não se ouvem):

- Frequência 915,0 MHz · largura de banda 125 kHz · SF7 · taxa de código 4/5 · *sync word* 0x12 · potência 14 dBm.
- A *sync word* funciona como um "canal privado": o rádio ignora pacotes de outras redes LoRa que usem outra palavra.
- Se o alcance dentro do prédio falhar, subir para SF9 (mais alcance, mais lento).
- Nunca ligar o módulo sem antena: a energia de transmissão volta para o chip e pode queimá-lo.

Por que um pacote binário e não JSON no LoRa? O LoRa transmite devagar; quanto menor o pacote, menos tempo no ar e menos chance de colisão. 11 bytes binários equivalem a uns 90 bytes em JSON. O gateway converte para JSON só depois, no Wi-Fi.

Byte	Campo	Descrição
0	versao	versão do protocolo (1); permite mudar o formato no futuro
1	tipo	1 = PONTO, 2 = ACK, 3 = CMD (comando do painel)
2–3	badge_id	ID do funcionário (crachá)
4–5	seq	número de sequência, cresce a cada ponto; o gateway descarta repetidos
6–9	hora	data e hora em "tempo Unix": segundos desde 01/01/1970, num inteiro de 4 bytes. No PONTO é a hora do relógio do crachá; no ACK é a hora certa do gateway, usada para acertar o relógio
10	flags	bit 0 = é retransmissão; bit 1 = ficou na fila (foi batido fora do alcance)

Por que tempo Unix? É um número só (ex.: 1793350935), fácil de mandar em 4 bytes e de converter para data em qualquer linguagem, sem confusão de formato dia/mês.

O próprio LoRa acrescenta um CRC (código de verificação) que detecta pacotes corrompidos.

Tópicos MQTT

Um tópico é como o endereço de uma caixa postal, organizado em níveis separados por `/`. O `+` e o `#` são curingas para assinar vários tópicos de uma vez (`unifesp/ponto/estacao/+/status` recebe o status de todas as estações).

Tópico	Quem publica → quem assina	Conteúdo
`unifesp/ponto/evento`	gateway e computador → Node-RED	ponto em JSON, com o campo `origem` (`botao` ou `ble`)
`unifesp/ponto/gateway/status`	gateway → Node-RED	`online` / `offline` e qualidade do LoRa

Tópico	Quem publica → quem assina	Conteúdo
`unifesp/ponto/pc/{badge}/status`	computador → Node-RED	crachá perto ou longe do computador (presença ao vivo)
`unifesp/ponto/cmd/{badge}`	Node-RED → gateway → crachá	comandos: `OTA`, `BIPE` (teste)

Dois recursos do MQTT que vamos usar: **retained** (o broker guarda a última mensagem do tópico e entrega na hora para quem assinar depois — o painel abre já sabendo o status) e **Last Will** (o gateway deixa um "testamento" no broker; se ele cair sem avisar, o broker publica `offline` por ele).

Exemplo de evento publicado:

```
{"ts":"2026-11-10T08:02:15-
03:00","badge":101,"origem":"botao","seq":42,"fila":false,"rssi_lora":-71,"sn
r":9.5}
{"ts":"2026-11-10T08:05:40-
03:00","badge":101,"origem":"ble","seq":7,"estado":"PRESENTE","rssi_ble":-58}
```

Modelo de dados (PostgreSQL)

O banco tem 3 tabelas. Uma tabela é como uma planilha: cada linha é um registro e cada coluna um campo. As tabelas se ligam por **chaves estrangeiras** (`REFERENCES`): o evento aponta para o funcionário, então o banco recusa um evento de crachá não cadastrado.

```
CREATE TABLE funcionario (
  id               SERIAL PRIMARY KEY,
  nome             TEXT NOT NULL,
  badge_id         INTEGER UNIQUE NOT NULL,
  carga_diaria_min INTEGER NOT NULL DEFAULT 480,  -- 8 h
  ativo            BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE evento (
  id            BIGSERIAL PRIMARY KEY,
  ts            TIMESTAMPTZ NOT NULL,
  badge_id      INTEGER NOT NULL REFERENCES funcionario(badge_id),
  origem        TEXT NOT NULL CHECK (origem IN ('botao','ble')),
  tipo          TEXT NOT NULL CHECK (tipo IN ('CHEGADA','SAIDA')),
  seq           INTEGER NOT NULL,
  rssi          SMALLINT,
```

```
  contabilizado BOOLEAN NOT NULL DEFAULT true,     -- false = ponto repetido
em menos de 30 min
  UNIQUE (badge_id, origem, seq)                    -- evita contar
retransmissão duas vezes
);

CREATE TABLE jornada (
  funcionario_id      INTEGER REFERENCES funcionario(id),
  data                DATE,
  primeira_chegada    TIMESTAMPTZ,
  ultima_saida        TIMESTAMPTZ,
  minutos_trabalhados INTEGER,
  saldo_min           INTEGER,                      -- trabalhado − carga
  PRIMARY KEY (funcionario_id, data)
);
```

- `TIMESTAMPTZ` guarda data e hora com fuso, evitando confusão com UTC.
- `UNIQUE (badge_id, origem, seq)` resolve as retransmissões: se o mesmo pacote chegar duas vezes, o segundo é recusado e o ponto não conta em dobro.
- `origem` guarda por onde o ponto chegou (botão/LoRa ou automático/BLE): o painel pode mostrar isso e o professor vê os dois caminhos funcionando.
- Regra de jornada: somar os intervalos de cada CHEGADA até a SAIDA seguinte no dia, só com eventos `contabilizado = true`; uma CHEGADA ainda sem SAIDA conta até o momento atual.

Por que PostgreSQL? É um banco robusto, usado em empresas, com tipos de data/hora bons e consultas poderosas (funções de janela como `LEAD()` calculam os intervalos chegada → saída direto em SQL). Roda num container Docker ao lado do Node-RED.

Placa de circuito impresso (PCI)

A PCI é uma placa-base onde a Heltec encaixa por barras de pinos fêmeas, com botão, LED, buzzer e um encaixe para o módulo de relógio DS1302. O mesmo desenho serve para o crachá e para o gateway: no gateway, o encaixe do DS1302 fica vazio.

Por que uma placa-base e não soldar tudo direto na Heltec? A Heltec continua removível (se queimar, troca-se só ela) e a PCI concentra as ligações que hoje estariam em fios soltos na protoboard. Fazer um desenho só e fabricar 2 cópias economiza tempo e dá uma placa reserva para teste.

Pinagem proposta (Heltec WiFi LoRa 32 V3)

A Heltec já usa vários pinos internamente, então eles não podem ser usados no nosso circuito:

- **GPIO 8 a 14:** rádio LoRa (SX1262).
- **GPIO 17, 18 e 21:** display OLED.
- **GPIO 19 e 20:** USB.
- **GPIO 0, 3, 45 e 46:** pinos de "boot" (*strapping*); se estiverem no nível errado ao ligar, a placa não inicia.
- **GPIO 1, 35, 36 e 37:** leitura da bateria, LED da placa e controle do Vext.

Função	GPIO	Explicação
Botão	7	liga ao GND quando apertado; usa o *pull-up* interno (resistor dentro do chip que mantém o pino em 3,3 V quando solto)
LED	6	em série com resistor de 220 Ω: (3,3 V − 2,0 V do LED vermelho) ÷ 220 Ω ≈ 6 mA, suficiente para acender bem sem forçar o pino
Buzzer	5	liga direto: terminal + do buzzer no GPIO 5 e o outro no GND. Em 3,3 V o buzzer de 5 V apita mais baixo, mas funciona. Se ficar fraco demais ou o ESP32 reiniciar ao apitar, entra um transistor BC547 com resistor de 1 kΩ (deixar o espaço dele previsto na PCI)
DS1302 CLK	40	relógio da comunicação com o módulo
DS1302 DAT	41	dados (vai e volta no mesmo fio)
DS1302 RST	42	habilita a comunicação (também chamado CE)
DS1302 VCC / GND	3V3 / GND	o módulo funciona de 2 a 5,5 V

Antes de desenhar a PCI: conferir cada pino no diagrama oficial de pinagem da Heltec V3 (site da Heltec, "WiFi LoRa 32 V3 pin map") e testar todos na protoboard. A tabela acima é uma proposta; a protoboard é que confirma.

Regras de projeto e o porquê de cada uma

- **KiCad** para esquemático e layout; tudo versionado em `hardware/` no Git.

- **Encaixe da Heltec:** a V3 tem 18 pinos em cada lado. Usar 2 barras fêmeas de 18 pinos (ver lista de materiais) com a distância exata entre as fileiras, medida na placa com paquímetro.
- **Encaixe do DS1302:** barra fêmea de 5 pinos na ordem VCC, GND, CLK, DAT, RST, igual à do módulo.
- **Face simples:** a placa de fenolite comprada tem cobre de um lado só, então todas as trilhas ficam numa camada. Onde duas trilhas precisarem se cruzar, usa-se um *jumper* (pedacinho de fio por cima da placa).
- **Fabricação por transferência térmica + percloreto de ferro** (ou fresadora, se o laboratório tiver): trilhas de no mínimo 0,8 mm, 0,5 mm de espaço entre elas e ilhas de 2 mm, porque o toner não reproduz detalhes finos.
- **Tamanho:** cerca de 6 × 8 cm. Cada placa de fenolite de 20 × 20 cm rende várias PCIs, então dá para errar e refazer.
- **Furos M3 nos cantos** para parafusar na caixa 3D.

Documentação da PCI (item obrigatório)

- [ ] Esquemático em PDF
- [ ] Layout (camadas de cobre e serigrafia) e render 3D gerado pelo KiCad
- [ ] Gerbers e arquivo de furação (os arquivos que a máquina usa para fabricar)
- [ ] Lista de componentes da placa
- [ ] Fotos da placa crua, montada e funcionando
- [ ] Registro de erros encontrados e correções (revisão A → B) — professores valorizam ver o processo

Lista de materiais (BOM)

Situação em 29/10 para 1 crachá + 1 gateway: todo o material obrigatório já está com o grupo ou no pedido do Mercado Livre, e a fabricação da PCI é feita com o material da universidade. Saem do projeto: ESP32 DevKit, ESP32-C3, módulo RFM95W, antenas avulsas, LED RGB e capacitores.

Bloco	Item	Qtd	Status	Para que serve / observação
Crachá e gateway	Heltec WiFi LoRa 32 V3 com antena	2	Já temos	#1 = crachá, #2 = gateway. Nunca ligar sem a antena: a energia do rádio volta e pode queimar o chip

Bloco	Item	Qtd	Status	Para que serve / observação
Crachá	Módulo relógio DS1302	1	Já temos	Mantém data e hora no crachá, mesmo desligado
Crachá	Bateria CR2032 3 V Philips (cartela com 5)	1 cartela	Comprando	Bateria do DS1302; as outras 4 ficam de reserva
Crachá e gateway	Botão táctil 6×6 mm	2	Já temos	Bater o ponto (crachá) e teste (gateway)
Crachá e gateway	LED 5 mm vermelho	2	Já temos	Erro ou ponto na fila
Crachá e gateway	LED 5 mm verde	2	Já temos	Ponto confirmado
Crachá e gateway	Buzzer	2	Já temos	Bipe de confirmação, ligado direto no pino do ESP32. Conferir se é ativo (ver abaixo)
Crachá e gateway	Resistores 220 Ω	4	Já temos	Um por LED (vermelho e verde). Conferir as faixas: vermelho-vermelho-marrom
Crachá e gateway	Transistor NPN BC547 + resistor 1 kΩ	2	Opcional	Orientação do professor: só comprar se o buzzer ligado direto apitar fraco demais ou fizer o ESP32 reiniciar
PCI	Placa de fenolite virgem 20×20 cm (kit com 3)	1 kit	Comprando	Matéria-prima da PCI; cada placa rende várias PCIs de 6×8 cm
PCI	Barra de pinos fêmea 1×20 (kit com 20)	1 kit	Comprando	Cortar em 18 pinos: 2 fileiras por placa × 2 placas. Uma barra cortada em 5 pinos vira o encaixe do DS1302; o resto fica de reserva

Bloco	Item	Qtd	Status	Para que serve / observação
PCI	Material de fabricação (percloreto ou fresadora, brocas, solda)	—	Laboratório	Fornecido pela universidade
Computador	Adaptador USB Bluetooth 5.0	1	Opcional	Só se o teste de compatibilidade mostrar que o PC não tem BLE
Geral	Cabo USB-C de dados	2	Já temos	Gravar e alimentar as Heltec. Conferir se não é cabo só de carga (esses não gravam)
Geral	Power bank (bateria de celular)	1	Opcional	Deixa o crachá portátil na demonstração: andar pelo prédio e bater o ponto à distância
Geral	Notebook (servidor) e celular (hotspot)	1 de cada	Já temos	Mosquitto, Node-RED e PostgreSQL rodam no notebook
Geral	Protoboard, jumpers, ferramentas de solda	—	Laboratório	Validação de circuito e montagem
Encapsulamento	Filamento PLA e impressora 3D	—	Laboratório	Caixas do crachá e do gateway (desafio extra)

Como saber se o buzzer é ativo ou passivo: o *ativo* tem um oscilador dentro e apita sozinho quando recebe tensão contínua; o *passivo* precisa que o microcontrolador gere a frequência (função `tone()`). Teste: ligar o terminal mais comprido (+) no 5 V e o outro no GND. Se apitar sem parar, é ativo. Se só der um estalo, é passivo. Os dois servem: muda apenas uma linha do código.

Desafios extras (+3 pontos)

Os três extras cabem no prazo se forem parte do sistema desde o início, e não acréscimos na última semana. O professor exige que funcionem "de maneira confiável" na demonstração, então cada um tem um teste definido.

1. Banco de dados para histórico (PostgreSQL)

- **Como:** container PostgreSQL no mesmo `docker-compose.yml`; no Node-RED, o nó `postgresql` (paleta `node-red-contrib-postgresql`) executa um `INSERT` a cada evento e `SELECT`s para o painel. As tabelas estão em Protocolos e dados.
- **Por que conta como desafio:** sem banco, os dados vivem só na memória do Node-RED e somem quando ele reinicia; com banco existe histórico de dias e semanas.
- **Evidência:** mostrar o histórico de dias anteriores; reiniciar o servidor ao vivo e mostrar que os dados continuam lá; exportar o espelho de ponto em CSV.

2. Atualização remota de firmware (OTA)

- **Como funciona por dentro:** a memória flash do ESP32 é dividida em duas áreas de programa (partições `app0` e `app1`). O firmware em uso roda de uma delas; a atualização é gravada na outra e, só se chegar inteira e correta, o ESP32 passa a iniciar por ela. Se algo der errado no meio, ele continua no firmware antigo. Por isso o esquema de partição "with OTA" é escolhido na Arduino IDE desde o primeiro firmware.
- **Gateway:** já está no Wi-Fi, então usa `ArduinoOTA`: a placa aparece na Arduino IDE como "porta de rede" e grava sem cabo.
- **Crachá:** LoRa é lento demais para transferir um firmware de ~1 MB. Então o painel envia só o comando `OTA` (MQTT → gateway → LoRa); o crachá liga o próprio Wi-Fi no hotspot, baixa o arquivo `.bin` (Sketch → Exportar binário compilado) de um servidor HTTP no notebook com `HTTPUpdate` e reinicia.
- **Evidência:** ao vivo, mudar a mensagem do OLED ou o número da versão, clicar "Atualizar" no painel e mostrar a nova versão no display.
- É o desafio com mais risco técnico; manter sempre o cabo USB como plano B.

3. Encapsulamento impresso em 3D

- **Como:** modelar as caixas no Tinkercad (mais fácil para iniciantes), Fusion 360 ou Onshape, a partir das medidas reais da PCI e dos módulos, tiradas com paquímetro.
- Crachá: rasgo para cordão, acesso à USB-C e à chave. Nó: furo para a antena SMA e janela para o LED. Gateway: janela para o display.
- Deixar folga de 0,3 a 0,5 mm nos encaixes: a impressora sempre deposita um pouco a mais de material. Imprimir um protótipo cedo para ajustar.

- **Evidência:** peças físicas montadas + arquivos STL/STEP no Git.

Divisão de tarefas (4 pessoas)

Cada pessoa é dona de um bloco e apoia outro, para que todos consigam explicar o sistema inteiro na arguição individual. Preencham os nomes.

Papel	Responsável	Entregas principais	Apoia
A — Hardware e PCI	(nome)	Circuito em protoboard (botão, LED, buzzer, DS1302), esquemático e layout no KiCad, fabricação e soldagem, documentação da PCI	D (caixas 3D usam as medidas da placa)
B — Firmware do crachá	(nome)	Botão com debounce, relógio DS1302, envio LoRa com ACK e fila, anúncio BLE, OLED	A (testes da placa)
C — Gateway e nuvem	(nome)	Firmware do gateway (LoRa → MQTT, ACK com hora, NTP, OLED), Mosquitto + Node-RED + PostgreSQL, OTA	B (protocolo LoRa)
D — Computador e integração	(nome)	Teste de compatibilidade BLE, agente Python (presença, bloqueio, ponto automático), painel no Node-RED, caixas 3D, Git e documentação	C (fluxos Node-RED)

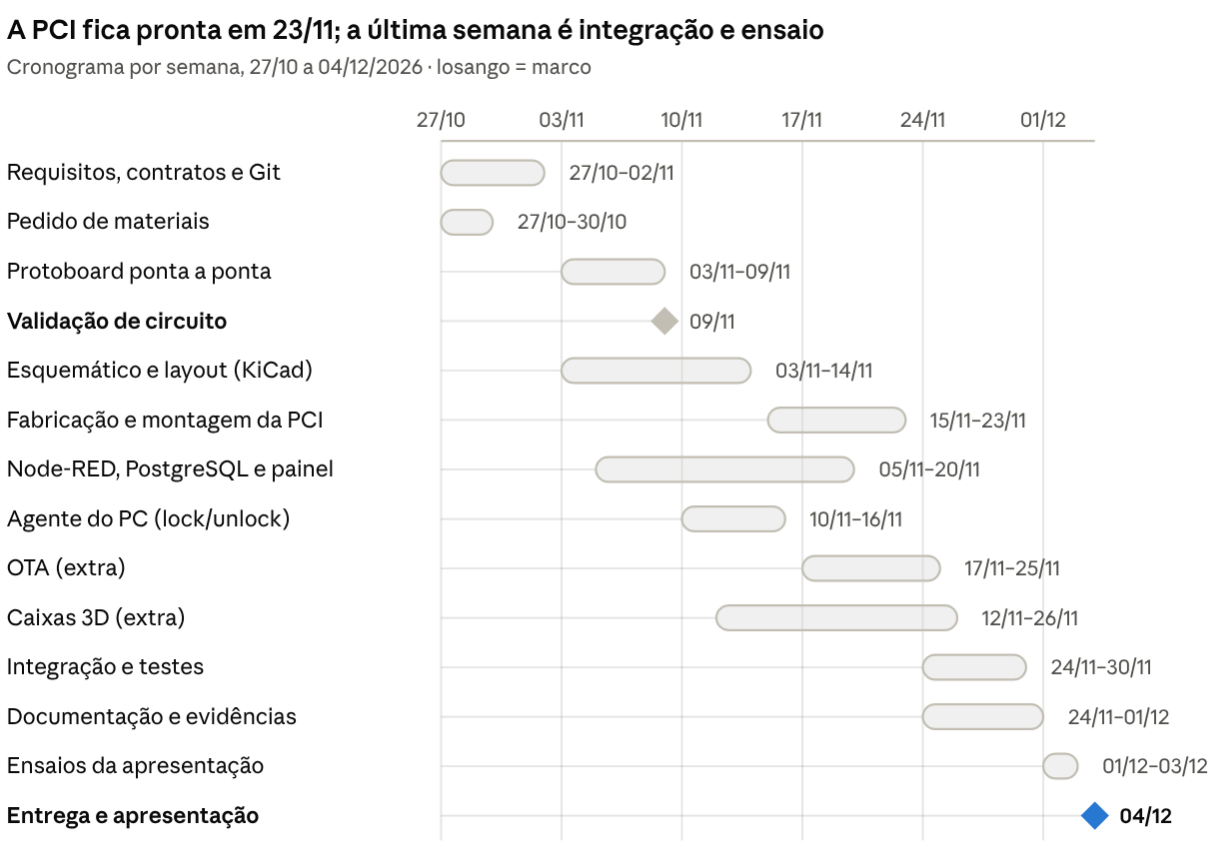
Regras de Git

- **Um repositório no GitHub** ("monorepo"), com uma pasta por parte: `firmware/badge`, `firmware/node`, `firmware/gateway` (sketches da Arduino IDE), `cloud/` (`docker-compose.yml`, `flows.json` exportado do Node-RED e o `schema.sql` do PostgreSQL), `pc-agent/`, `hardware/` (KiCad), `enclosure/` (STL/STEP) e `docs/`.
- **Commits pequenos e frequentes** com mensagem clara ("nó: adiciona ACK e retransmissão"). Um commit é uma "foto" do projeto naquele momento; se algo quebrar, dá para voltar à foto anterior.

- **Um branch por funcionalidade** (`feature/ack-lora`): um branch é uma cópia paralela onde se trabalha sem atrapalhar o código principal (`main`). Quando estiver funcionando, abre-se um **Pull Request**, outro membro revisa e o código entra na `main` .
- **Cada membro commita com a própria conta:** o histórico do Git vira a prova da contribuição individual que a apresentação exige.
- **Nunca subir senhas:** o `config.h` com a senha do Wi-Fi fica no `.gitignore` (arquivo que lista o que o Git deve ignorar); no repositório vai só o `config.example.h` .
- Para quem nunca usou: o GitHub Desktop faz commits, branches e PRs com botões, sem linha de comando.

Cronograma até 04/12

O caminho crítico é a PCI: o layout precisa estar pronto em 14/11 para fabricar e montar até 23/11, deixando a última semana para integração, documentação e ensaio.



cronograma · 27/10 a 04/12

Nuvem, agente do PC e caixas 3D correm em paralelo com a PCI, cada um com seu dono; o marco de 09/11 (circuito validado em protoboard) é o ponto de controle para saber se o plano está em dia.

Riscos, decisões em aberto e próximos passos

Risco	Impacto	Mitigação
Computador sem Bluetooth 4.0 (BLE)	Ponto automático e desbloqueio não funcionam	Fazer o teste de compatibilidade esta semana; adaptador USB Bluetooth 5.0 como plano B
Rede da faculdade com login ou isolamento entre aparelhos	Gateway não chega ao servidor	Usar o hotspot do celular em 2,4 GHz
Barras fêmeas de 15 pinos não encaixam a Heltec (18 pinos)	PCI sem encaixe	Resolvido: compradas barras 1×20, cortadas em 18
Bateria CR2016 frouxa no DS1302	Relógio perde a hora ao desligar	Resolvido: compradas CR2032
RSSI do BLE oscila e gera bloqueio falso	Demo falha na frente do professor	Média móvel + histerese + tempo mínimo; calibrar no laboratório real
Atraso na fabricação da PCI	Perde 3 pontos	Layout pronto até 14/11; 2 placas para ter reserva
Windows não permite desbloqueio programático	Desbloqueio parcial	Demo em Linux, ou tela de bloqueio do próprio agente
OTA corrompe o firmware	Crachá para de funcionar na demo	Partições OTA com rollback; gravar por USB como plano B

Decisões em aberto

- [ ] O computador da demonstração roda Linux ou Windows? Define como o agente desbloqueia a tela.
- [ ] O computador tem Bluetooth 4.0+? (resultado do teste de compatibilidade)
- [x] ~~Material para fabricar a PCI: resolvido, a universidade fornece.~~
- [ ] Vamos tentar ligar o PC por Wake-on-LAN? Só se o PC estiver no cabo de rede, na mesma rede do gateway.
- [ ] Um crachá só ou dois? Dois exigiriam uma terceira placa LoRa.

Próximos passos desta semana

- [x] ~~Ajustar o pedido no Mercado Livre: CR2032 no lugar da CR2016 e barra fêmea 1×20 no lugar da 1×15~~
- [ ] Preencher os nomes na divisão de tarefas e criar o repositório no GitHub
- [ ] Teste de compatibilidade BLE no computador que será usado
- [ ] Teste 1: ping-pong LoRa entre as duas Heltec (com antenas!)
- [ ] Teste 2: ler e acertar a hora do DS1302 na protoboard
- [ ] Teste 3: botão + LED + buzzer na protoboard
- [ ] Instalar Arduino IDE + pacote ESP32 e subir Mosquitto, Node-RED e PostgreSQL; publicar uma mensagem de teste pelo hotspot